

T.C.
SAKARYA ÜNİVERSİTESİ

Bilgisayar ve Bilişim Bilimleri Fakültesi
Bilgisayar Mühendisliği Bölümü



PROJE KONUSU
TARSAU - Basit Arşivleme Programı

Sistem Programlama 2025-2026 Bahar Dönemi Projesi

ÖĞRENCİNİN ADI-SOYADI:AWAB ATA
NUMARASI:B211210586

1. Proje hakkında:

Bu projede, `tar`, `rar` veya `zip` gibi çalışan ancak sıkıştırma yapmayan "tarsau" adlı bir arşivleme programı geliştirilmiştir. Program, Linux/Unix işletim sisteminde C dilinde yazılmış olup komut satırından çalıştırılmaktadır.

1.1 Özellikler

Özellik	Açıklama
Arşivleme (`-b`)	Metin dosyalarını `.sau` formatında birleştirir
Çıkarma (`-a`)	`.sau` arşiv dosyasından dosyaları
Dosya İzinleri	Arşivlenen dosyaların izinleri korunur
Hata Yönetimi	Tüm hata durumları uygun mesajlarla ele alınır
Makefile	Proje `make` komutuyla derlenir

1.2 Kısıtlamalar

- Giriş dosyaları yalnızca ASCII metin dosyaları olabilir
- Maksimum dosya sayısı: 32
- Maksimum toplam boyut: 200 MB

2. Geliştirme Süreci

Proje, git versiyon kontrol sistemi kullanılarak adım adım geliştirilmiştir. Geliştirme süreci aşağıdaki temel aşamalardan oluşmaktadır:

1. Temel Yapı ve Makefile (12 Mayıs): Projenin başlangıç dosyaları (`tarsau.c`, `Makefile`, `.gitignore`) oluşturuldu. Argüman okuma ve temel hata kontrolleri eklendi.
2. Yardımcı Fonksiyonlar (13 Mayıs): Dosyaların ASCII olup olmadığını kontrol eden, dosya boyutunu bulan ve dosya izinlerini (`rwxrwxrwx` formatında) okuyup dönüştüren fonksiyonlar implement edildi.
3. Arşivleme Modu -b (14 Mayıs): `.sau` arşiv dosyasının organizasyon bölümünü ve içerik kısmını kurallara uygun (10 bayt boyut bilgisi, `|` ayracı, vb.) şekilde oluşturan fonksiyonlar yazıldı.
4. Arşivden Çıkarma Modu -a (15 Mayıs): Arşiv dosyasını okuyup ayrıştıran, gerekli izinleri oluşturan ve dosyaları orijinal izinleriyle geri çıkaran kodlar eklendi.
5. Test ve Hata Düzeltmeleri (16 Mayıs): Sınır durumlar (büyük dosyalar, bozuk arşivler, ASCII olmayan dosyalar) test edildi ve bellek sızıntıları kontrol edildi.

6. Raporlama (17 Mayıs): Geliştirme süreci, kullanım detayları ve kod açıklamalarını içeren bu rapor hazırlandı.

3. Kullanım :

3.1 Derleme

- bash
- make

3.2 Arşivleme (Dosyaları Birleştirme)

bash

Varsayılan çıktı (a.sau)

```
./tarsau -b dosya1.txt dosya2.txt dosya3.txt
```

Belirli çıktı dosyası

```
./tarsau -b t1 t2 t3 t4.txt t5.dat -o arxiv.sau
```

*Örnek Çıktı: Dosyalar birleştirildi.

3.3 Arşivden Çıkarma

Bash

Geçerli dizine çıkar

```
./tarsau -a arxiv.sau
```

Belirli bir dizine çıkar

```
./tarsau -a arxiv.sau hedef_dizin
```

Örnek Çıktı:

hedef_dizin dizininde t1, t2, t3, t4.txt, t5.dat dosyaları acildi.

4. .sau Arşiv Dosyası Formatı:

Arşiv dosyası iki ana bölümden oluşur:

4.1 Format Yapısı:

10 Bayt	Organizasyon Bölümü	Dosya İçerikleri	
(Boyut)	dosya_adi,izinler,boyut ...	(Art arda)	

4.2 Organizasyon (İçerik) Bölümü:

- İlk 10 bayt: Organizasyon bölümünün toplam boyutunu ASCII formatında içerir
- Kayıtlar: karakteri ile ayrılır
- Alanlar: Virgülle ayrılmış: dosya_adi,izinler,boyut

Örnek:

```
0000000062|test1.txt,rw-r--r--,35||test2.txt,rw-r--r--,42||test3.txt,rw-r--r--,28|
```

4.3 Arşivlenmiş Dosyalar Bölümü:

- Dosya içerikleri ayırıcı kullanılmadan art arda yerleştirilir
- Her dosyanın boyutu organizasyon bölümünden okunarak dosya sınırları belirlenir

5. Kaynak Kod Açıklaması:

5.1 Dosya Yapısı:

proje/

— tarsau.c	# Ana kaynak kod
— Makefile	# Derleme dosyası
└— RAPOR.md	# Bu rapor

5.2 Temel Fonksiyonlar:

main() - Giriş Noktası

Komut satırı argümanlarını ayrıştırır ve ilgili modu (-b veya -a) çağırır.

c

```
int main(int argc, char *argv[])
{
    if (argc < 2) {
        // Kullanım bilgisi göster

        return 1;
    }

    if (strcmp(argv[1], "-b") == 0) {
        // Arşivleme modu

        return arşivle(dosya_sayisi, dosya_adlari, arşiv_adi);
    }

    else if (strcmp(argv[1], "-a") == 0) {
        // Çıkarma modu

        return arşivden_cikar(arşiv_adi, dizin_adi);
    }
}
```

metin_dosyasi_mi() - ASCII Doğrulama

Dosyanın geçerli bir ASCII metin dosyası olup olmadığını kontrol eder. Her baytın 0-127 aralığında olmasını sağlar.

c

```
static int metin_dosyasi_mi(const char *dosya_adi)

{

    FILE *fp = fopen(dosya_adi, "rb");

    int ch;

    while ((ch = fgetc(fp)) != EOF) {

        if (ch > 127) {

            fclose(fp);

            return 0; // ASCII degil

        }

    }

    fclose(fp);

    return 1; // Gecerli ASCII

}
```

arsivle() - Arşivleme İşlemi:

1. Tüm giriş dosyalarını doğrular (varlık, format, boyut)
2. Organizasyon bölümünü oluşturur
3. Arşiv dosyasını yazar (10 bayt boyut + org bölümü + dosya içerikleri)

c

```
static int arshivle(int dosya_sayisi, char *dosya_adlari[], const char *arsiv_adi)
```

```

{

// 1. Dogrulama

for (i = 0; i < dosya_sayisi; i++) {

    if (!metin_dosyasi_mi(dosya_adlari[i])) {

        fprintf(stderr, "%s giris dosyasinin formati uyumsuzdur!\n", dosya_adlari[i]);

        return 1;

    }

}

}

// 2. Organizasyon bolumunu olustur

for (i = 0; i < dosya_sayisi; i++) {

    snprintf(org_bolumu + org_pos, ..., "|%s,%s,%ld|",

        dosya_adlari[i], izinler[i], boyutlar[i]);

}

// 3. Arsiv dosyasini yaz

fprintf(arsiv_fp, "%010d", org_boyutu); // 10 bayt boyut

fwrite(org_bolumu, 1, org_boyutu, arsip_fp); // Org bolumu

// Dosya iceriklerini art arda yaz

}

```

arsivden_cikar() - Çıkarma İşlemi

1. Arşiv dosyasını açar ve doğrular
2. Organizasyon bölümünü okur ve ayrıştırır
3. Hedef dizini oluşturur
4. Dosyaları çıkarır ve izinleri ayarlar

c

```
static int arsvden_cikar(const char *arsiv_adi, const char *dizin_adi)
```

```
{
```

```
    // 1. İlk 10 bayti oku
```

```
    fread(boyut_str, 1, 10, arsv_fp);
```

```
    int org_boyutu = atoi(boyut_str);
```

```
    // 2. Org bolumunu oku ve ayristir
```

```
    // |dosya_adi,izinler,boyut| formatinda kayitlar
```

```
    // 3. Dizin olustur (gerekliyorsa)
```

```
    dizin_olustur_recursive(dizin_adi);
```

```
    // 4. Dosyalari cikar ve izinleri ayarla
```

```
    for (i = 0; i < dosya_sayisi; i++) {
```

```
        // Dosya icerigini yaz
```

```
        // chmod ile izinleri ayarla
```

```
}
```


izin_str_olustur() / izin_str_coz() - İzin Yönetimi

Dosya izinlerini rwxrwxrwx formatında saklar ve geri çözer.

c

```
static void izin_str_olustur(mode_t mode, char *buf)
```

```
{  
  
    buf[0] = (mode & S_IRUSR) ? 'r' : '-';  
  
    buf[1] = (mode & S_IWUSR) ? 'w' : '-';  
  
    buf[2] = (mode & S_IXUSR) ? 'x' : '-';  
  
    // ... (grup ve diğer izinleri)  
  
}
```

```
static mode_t izin_str_coz(const char *izin_str)
```

```
{  
  
    mode_t mode = 0;  
  
    if (izin_str[0] == 'r') mode |= S_IRUSR;  
  
    if (izin_str[1] == 'w') mode |= S_IWUSR;  
  
    // ...  
  
    return mode;  
  
}
```

6. Hata Yönetimi:

Program aşağıdaki hata durumlarını ele alır:

Hata Durumu	Mesaj
Dosya bulunamadı	'dosya' dosyasi bulunamadi!
ASCII olmayan dosya	dosya giris dosyasinin formati uyumsuzdur
200 MB sınırı aşıldı	Giris dosyalarinin toplam boyutu 200 MB'i geciyor!
32 dosya sınırı aşıldı	En fazla 32 dosya arsilenebilir
Bozuk arşiv dosyası	Arsiv dosyasi uygunsuz veya bozuk!
Dizin oluşturma hatası	'dizin' dizini olusturulamadi

7. Test Sonuçları :

7.1 Arşivleme Testi:

bash

```
$ echo "Merhaba Dunya!" > t1.txt
```

```
$ echo "Test dosyasi 2" > t2.txt
```

```
$ echo "Ucuncu dosya" > t3.txt
```

```
$ ./tarsau -b t1.txt t2.txt t3.txt -o test.sau
```

Dosyalar birlestirildi.

7.2 Arşivden Çıkarma Testi:

bash

```
$ ./tarsau -a test.sau cikarilan
```

cikarilan dizininde t1.txt, t2.txt, t3.txt dosyalari acildi.

```
$ diff t1.txt cikarilan/t1.txt # Fark yok
```

```
$ diff t2.txt cikarilan/t2.txt # Fark yok
```

```
$ diff t3.txt cikarilan/t3.txt # Fark yok
```

7.3 Hata Durumu Testleri :

```
bash
```

```
$ ./tarsau -b resim.png -o test.sau
```

resim.png giris dosyasinin formati uyumsuzdur!

```
$ ./tarsau -a bozuk.txt
```

Arsiv dosyasi uygunsuz veya bozuk!

8. Derleme ve Çalıştırma

```
bash
```

```
# Derleme
```

```
Make
```

```
# Temizleme
```

```
make clean
```

Otomatik test

make test

Test dosyalarını temizle

make testclean

9. GitHub Deposu

Proje geliştirme süreci aşağıdaki GitHub adresi üzerinden yürütülmüştür:

> GitHub: <https://github.com/Awab21-joke/tarsau>

Aşağıdaki ekran çıktısında programın başarıyla derlendiği (make), hata durumlarının başarıyla yakalandığı ve arşivleme/çıkarma işlemlerinin sorunsuz çalıştığı görülmektedir:

```
System information as of Mon May 18 19:35:57 +03 2026
System load:  0.54      Processes:           55
Usage of /:   0.2% of 1006.85GB   Users logged in:    0
Memory usage: 9%      IPv4 address for eth0: 172.31.248.70
Swap usage:   0%

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/awab/.hushlogin file.
awab@Dammnnn:/mnt/c/Users/USER/OneDrive/Resimler/SISTEM PROGLAMA PROJESI$ make
gcc -Wall -Wextra -std=c99 -pedantic -o tarsau tarsau.c
awab@Dammnnn:/mnt/c/Users/USER/OneDrive/Resimler/SISTEM PROGLAMA PROJESI$ ./tarsau -b t1.txt t2.txt -o test_arsiv.sau
t1.txt giriş dosyasının formatı uyumsuzdur!
awab@Dammnnn:/mnt/c/Users/USER/OneDrive/Resimler/SISTEM PROGLAMA PROJESI$ ./tarsau -a test_arsiv.sau cikti_klasoru
Arşiv dosyası uygunsuz veya bozuk!
awab@Dammnnn:/mnt/c/Users/USER/OneDrive/Resimler/SISTEM PROGLAMA PROJESI$ ls -la cikti_klasoru
ls: cannot access 'cikti_klasoru': No such file or directory
awab@Dammnnn:/mnt/c/Users/USER/OneDrive/Resimler/SISTEM PROGLAMA PROJESI$ echo "Merhaba bu birinci dosya" > t1.txt
echo "Bu da ikinci dosya" > t2.txt
./tarsau -b t1.txt t2.txt -o test_arsiv.sau
./tarsau -a test_arsiv.sau cikti_klasoru
ls -la cikti_klasoru
Dosyalar birleştirildi.
cikti_klasoru dizininde t1.txt, t2.txt dosyaları acildi.
total 0
drwxrwxrwx 1 awab awab 4096 May 18 19:36
drwxrwxrwx 1 awab awab 4096 May 18 19:36
-rwxrwxrwx 1 awab awab 25 May 18 19:36 t1.txt
-rwxrwxrwx 1 awab awab 19 May 18 19:36 t2.txt
awab@Dammnnn:/mnt/c/Users/USER/OneDrive/Resimler/SISTEM PROGLAMA PROJESI$
```


